

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
CAMPUS CURITIBA
BACHARELADO EM ENGENHARIA DE SOFTWARE

ARTHUR RODRIGUES PANSERA
JEAN INÁCIO PRAES MOURA
JOÃO GABRIEL DE LIMA COLTRE
STEFANY CARLOS DE OLIVEIRA

BLUEFLOW: MONITORADOR DE CONSUMO DE ÁGUA

CURITIBA
2025

ARTHUR RODRIGUES PANSERA
JEAN INÁCIO PRAES MOURA
JOÃO GABRIEL DE LIMA COLTRE
STEFANY CARLOS DE OLIVEIRA

BLUEFLOW: MONITORADOR DE CONSUMO DE ÁGUA

Trabalho apresentado ao curso de Bacharelado em Engenharia de Software da Pontifícia Universidade Católica do Paraná – Campus Curitiba, como requisito parcial da disciplina Performance em Sistemas Ciberfísicos.

Área de concentração: Sistemas embarcados e IoT.

Orientador: Prof. Fabio Garcez Bettio

CURITIBA

2025

SUMÁRIO

1 INTRODUÇÃO	4
2 DESENVOLVIMENTO	4
2.1 Primeira Etapa (TDE01) – Apresentação da Proposta	5
2.1.1 Objetivo do Projeto	5
2.1.2 Justificativa.....	5
2.1.3 Tecnologias/Ferramentas Utilizadas	6
2.1.4 Arquitetura Geral do Sistema	11
2.1.4.1 Diagrama Simplificado do Sistema.....	12
2.1.4.2 Diagrama de Ligação Eletrônica Geral	13
2.1.4.3 Diagrama de Fluxos	14
2.1.4 Cronograma de Execução.....	15
2.1.5 Testes Isolados de Sensores, Atuadores e Módulos	16
2.2 Segunda Etapa (TDE02) – Documentação Técnica.....	23
2.2.1 Especificação do Sistema	23
2.2.2 Diagrama de Arquitetura Ciberfísica	25
2.2.3 Código-Fonte Documentado.....	26
2.2.3.1 Código para Leitura de Vazão e Envio ao Firebase (monitoramento-consumo-de-agua.ino)	27
2.2.3.2 Código JavaScript para Processamento, Atualização e Visualização dos Dados de Vazão (script.js)	29
2.2.3.3 Código da Interface Web para Visualização de Consumo (index.html).....	35
2.2.3.4 Código para Estilização da Interface Web (style.css)	37
2.2.4 Relatório de Desempenho e Testes.....	41
3 CONSIDERAÇÕES FINAIS	42
4 LINKS (REPOSITÓRIO GITHUB E SIMULAÇÃO WOKWI).....	43
5 REFERÊNCIAS BIBLIOGRÁFICAS	44

1 INTRODUÇÃO

Problema escolhido: Monitoramento de consumo de água.

O uso consciente da água desempenha papel crucial na manutenção dos recursos naturais e na promoção do desenvolvimento sustentável. Atualmente, o desperdício e a ausência de monitoramento eficiente dificultam a gestão adequada desse recurso vital, resultando em sérios impactos ambientais, econômicos e sociais. Com a crescente urbanização e a demanda por recursos limitados, torna-se essencial implementar soluções tecnológicas que possibilitem o controle preciso do consumo doméstico.

Nesse sentido, tecnologias embarcadas, como o microcontrolador ESP32, combinadas a plataformas em nuvem, vêm se destacando por oferecer soluções acessíveis e eficientes para o monitoramento em tempo real. Esses sistemas permitem não apenas a medição contínua do consumo, mas também a análise de padrões que auxiliam na identificação de possíveis vazamentos, contribuindo para a economia e a sustentabilidade.

Este trabalho apresenta o desenvolvimento de um sistema para monitorar o consumo de água, com foco na medição precisa do volume utilizado e na detecção de vazamentos. A integração entre sensores, comunicação Wi-Fi e armazenamento em nuvem visa proporcionar aos usuários maior controle sobre seu consumo, promovendo a conscientização e incentivando práticas mais sustentáveis no uso da água.

2 DESENVOLVIMENTO

Esta seção descreve o desenvolvimento do projeto, detalhando as etapas realizadas desde a definição da proposta até sua implementação e testes. O trabalho foi dividido em duas etapas principais: a primeira voltada à concepção e planejamento do sistema, e a segunda à documentação técnica e análise de desempenho. A seguir, são apresentados os objetivos, justificativas, ferramentas utilizadas, cronograma e demais aspectos relevantes.

2.1 Primeira Etapa (TDE01) – Apresentação da Proposta

Nesta etapa, a equipe definiu o projeto com seus objetivos, justificativas, tecnologias, arquitetura e cronograma. Também foram feitos testes individuais nos sensores e módulos para garantir seu funcionamento antes da integração.

2.1.1 Objetivo do Projeto

Desenvolver um sistema de monitoramento de consumo de água utilizando o microcontrolador ESP32, com capacidade de medir em tempo real o volume de água utilizado por meio de um sensor de fluxo. Os dados serão enviados via Wi-Fi para o Firebase Realtime Database, possibilitando sua visualização remota. O sistema também incluirá uma funcionalidade de detecção de vazamentos, identificando padrões anormais de consumo.

2.1.2 Justificativa

O uso consciente da água tornou-se uma necessidade urgente diante da crescente escassez hídrica e do elevado índice de desperdício. De acordo com o estudo “Perdas de Água 2024 (SNIS 2022)” divulgado pelo Instituto Trata Brasil, aproximadamente 37,8% da água tratada no país foi perdida antes de chegar às residências, o que representa cerca de 7 bilhões de metros cúbicos desperdiçados, volume suficiente para abastecer toda a população do Rio Grande do Sul por mais de cinco anos. Esse cenário se agrava ainda mais quando se considera que cerca de 32 milhões de brasileiros ainda não têm acesso à água potável, evidenciando a desigualdade no fornecimento e a necessidade urgente de soluções mais eficientes e sustentáveis.

Essa realidade também se reflete no ambiente doméstico, onde o consumo médio de água no Brasil é de cerca de 154 litros por habitante/dia, valor significativamente superior ao limite recomendado pela Organização das Nações Unidas (ONU), que é de 110 litros por pessoa/dia. Além do consumo excessivo, a ausência de ferramentas de monitoramento em tempo real dificulta a detecção imediata de vazamentos e de práticas ineficientes, o que contribui para o agravamento do desperdício e compromete os esforços de preservação.

Diante desse cenário, este projeto propõe uma solução prática, acessível e tecnológica: um sistema de monitoramento remoto do consumo de água por meio da internet. Utilizando o microcontrolador ESP32, o sistema realiza medições precisas do volume utilizado e identifica possíveis vazamentos a partir da análise dos padrões de uso. Ao tornar o consumo mais transparente e rastreável, a iniciativa visa promover a conscientização dos usuários, incentivar hábitos mais responsáveis e contribuir efetivamente para a economia de água, a redução dos custos na conta mensal e a sustentabilidade ambiental, além de oferecer uma solução prática de gestão de recursos no ambiente doméstico.

2.1.3 Tecnologias/Ferramentas Utilizadas

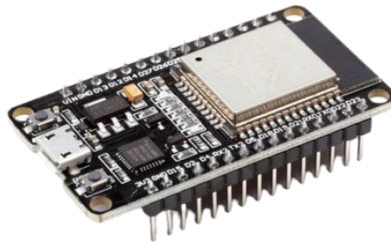
Tabela 1 - Tecnologias/Ferramentas Utilizadas

Nº	Categoria	Item/Descrição
1	Microcontrolador	ESP32 DevKit V1
2	Sensor	Sensor de fluxo de água YF-S201
3	Conectividade	Wi-Fi integrado ao ESP32
4	Plataforma na nuvem	Firebase Realtime Database
5	Simulação	Wokwi
6	Desenvolvimento	Arduino IDE, Visual Studio Code (VS Code)
7	Linguagens	C++, JavaScript, HTML/CSS
8	Bibliotecas	Wifi.h, FirebaseESP32.h, Ticker.h, Wire.h, time.h
9	Materiais auxiliares	Protoboard, jumpers, mangueira de jardim, cabo micro USB, adaptadores de mangueira

Fonte: Elaborado pela equipe.

- 1) O ESP32 é um microcontrolador de baixo custo que possui Wi-Fi e Bluetooth integrados. Ele será o "cérebro" do sistema, responsável por processar os dados coletados pelo sensor de fluxo de água e enviar essas informações para o Firebase via Wi-Fi.

Figura 1 - ESP32 DevKit V1.



Fonte: Victor Vision (2023).

- 2) O YF-S201 é um sensor de fluxo que permite medir o volume de água que passa por ele. Ele gera pulsos cada vez que uma certa quantidade de água passa, e cada pulso é contado pelo ESP32 para calcular o volume de água consumido. Esse sensor é simples e eficiente para aplicações de monitoramento de fluxo.

Figura 2 - Sensor de fluxo de água YF-S201.



Fonte: Eletruscomp.

- 3) O Wi-Fi integrado no ESP32 permite que ele se conecte a redes sem fio e envie os dados coletados para a nuvem (Firebase). Essa conectividade é fundamental para o monitoramento remoto do consumo de água, o que possibilita o acesso em tempo real a partir de qualquer lugar.

- 4) O Firebase é uma plataforma de desenvolvimento de aplicativos móveis e web, e o Realtime Database permite armazenar e sincronizar dados em tempo real. No projeto, ele será usado para armazenar as informações de consumo de água, que podem ser acessadas remotamente.
- 5) O Wokwi é uma plataforma online para simulação de circuitos eletrônicos, utilizada para testar virtualmente o sistema antes da montagem física. Apesar de os dados de consumo de água serem simulados, é possível validar toda a lógica de funcionamento, incluindo a comunicação com a nuvem e o processamento das informações pelo ESP32.
- 6) Ambientes de desenvolvimento:
- Arduino IDE: Ambiente de desenvolvimento integrado (IDE) utilizado para escrever, compilar e carregar o código no ESP32. É amplamente adotado em sistemas embarcados devido à sua simplicidade, compatibilidade com diversas bibliotecas e facilidade de uso.
 - Visual Studio Code (VS Code): Ferramenta de edição de código leve e poderosa, empregada na criação da interface web. Compatível com várias linguagens e extensões, perfeita para HTML, CSS e JavaScript.
- 7) Linguagens utilizadas:
- C++: Linguagem utilizada na programação do ESP32. Permite controle direto do hardware, sendo ideal para aplicações embarcadas e integração com sensores e bibliotecas específicas.
 - JavaScript: Empregado na criação da lógica da interface web, permitindo a manipulação dinâmica e manipulação dos dados para obtenção de diferentes informações.
 - HTML/CSS: O HTML foi utilizado para estruturar a interface web e o CSS para sua estilização, proporcionando uma visualização organizada e intuitiva dos dados enviados pelo ESP32.

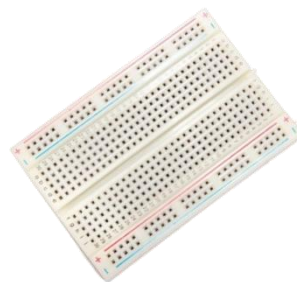
8) Bibliotecas utilizadas:

- Wifi.h: Responsável por habilitar a conectividade Wi-Fi do ESP32, permitindo que ele se conecte a redes sem fio.
- FirebaseESP32.h: Permite a integração do ESP32 com o Firebase, possibilitando o envio e a leitura de dados em tempo real pela internet.
- Ticker.h: Utilizada para agendar tarefas em intervalos específicos, o que facilita o controle do tempo entre leituras do sensor.
- Wire.h: Biblioteca que implementa a comunicação via protocolo I2C, frequentemente usada na integração com sensores e outros dispositivos periféricos.
- time.h: Usada para obter, configurar e manipular informações de tempo no ESP32, especialmente com servidores NTP, permitindo o registro de horários corretos mesmo sem um módulo de relógio físico (RTC).

9) Materiais auxiliares utilizados:

- Protoboard: Usada para montar o circuito de forma temporária, permitindo conexões rápidas e sem a necessidade de soldagem.

Figura 3 – Protoboard de 400 pontos.



Fonte: Maker Hero.

- Jumpers: Fios que conectam os componentes na protoboard.

Figura 4 - Jumpers Macho/Fêmea.



Fonte: Maker Hero.

- Mangueira de jardim: Usada para conectar o sensor de fluxo à tubulação de água.

Figura 5 - Mangueira de jardim siliconada.



Fonte: Casa das Correias e Borrachas.

- Adaptadores de mangueira: Têm o intuito de conectar a mangueira ao sensor, assegurando vedação e evitando vazamentos.

Figura 6 - Adaptador de mangueira.



Fonte: Top Opções.

- Cabo micro USB: Fornece energia ao ESP32 e permite a comunicação com o computador.

Figura 7 - Cabo micro USB.



Fonte: lbyte.

2.1.4 Arquitetura Geral do Sistema

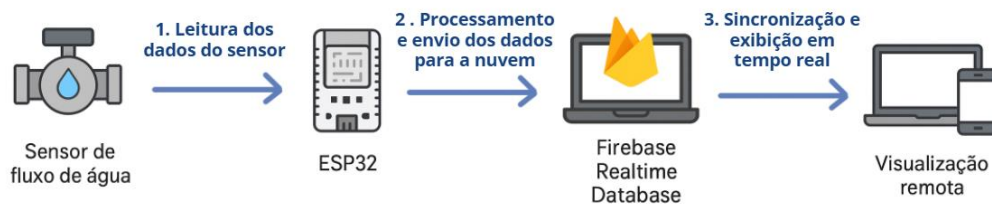
O sistema é composto por um ESP32, que coleta os dados do sensor de fluxo YF-S201. Esse sensor gera pulsos proporcionais ao volume de água que passa por ele, os quais são utilizados para calcular o consumo em litros.

O ESP32 processa os dados e os envia, via Wi-Fi, para o Firebase Realtime Database, onde ficam armazenados. Um cliente web ou aplicativo pode acessar esses dados remotamente para visualizar o consumo em tempo real. Além disso, o sistema analisa os padrões de uso: caso o consumo continue constante durante horários de inatividade (como durante a madrugada), um possível vazamento é identificado e um alerta pode ser enviado ao usuário.

Os usuários poderão acessar os dados por meio de uma interface web intuitiva, disponível em qualquer dispositivo com internet. Isso possibilita o monitoramento do consumo de água em tempo real, a qualquer hora e em qualquer lugar. A plataforma proporciona uma visualização clara, incentivando a conscientização sobre o uso da água e ajudando na identificação de excessos ou vazamentos.

2.1.4.1 Diagrama Simplificado do Sistema

Figura 8 - Diagrama Simplificado do Sistema.

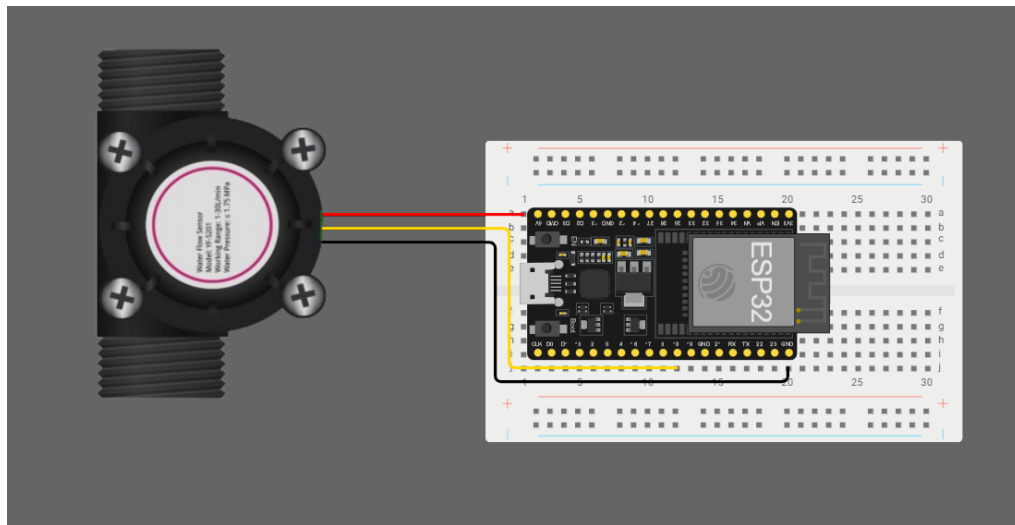


Fonte: Elaborado pela equipe.

- Sensor de Fluxo YF-S201:
 - Detecta o volume de água consumido, gerando pulsos de sinal que são lidos pelo ESP32.
- ESP32:
 - O microcontrolador que processa os sinais do sensor de fluxo e calcula o volume consumido.
 - Ele também realiza a comunicação via Wi-Fi com o Firebase, enviando os dados em tempo real.
- Firebase Realtime Database:
 - Armazena os dados de consumo e vazamento.
 - Permite o acesso remoto via interface web.
- Interface Web/App:
 - Usuários podem visualizar os dados em tempo real.
 - Fornece gráficos, alertas e relatórios de consumo.

2.1.4.2 Diagrama de Ligação Eletrônica Geral

Figura 9 - Diagrama Simplificado do Sistema.

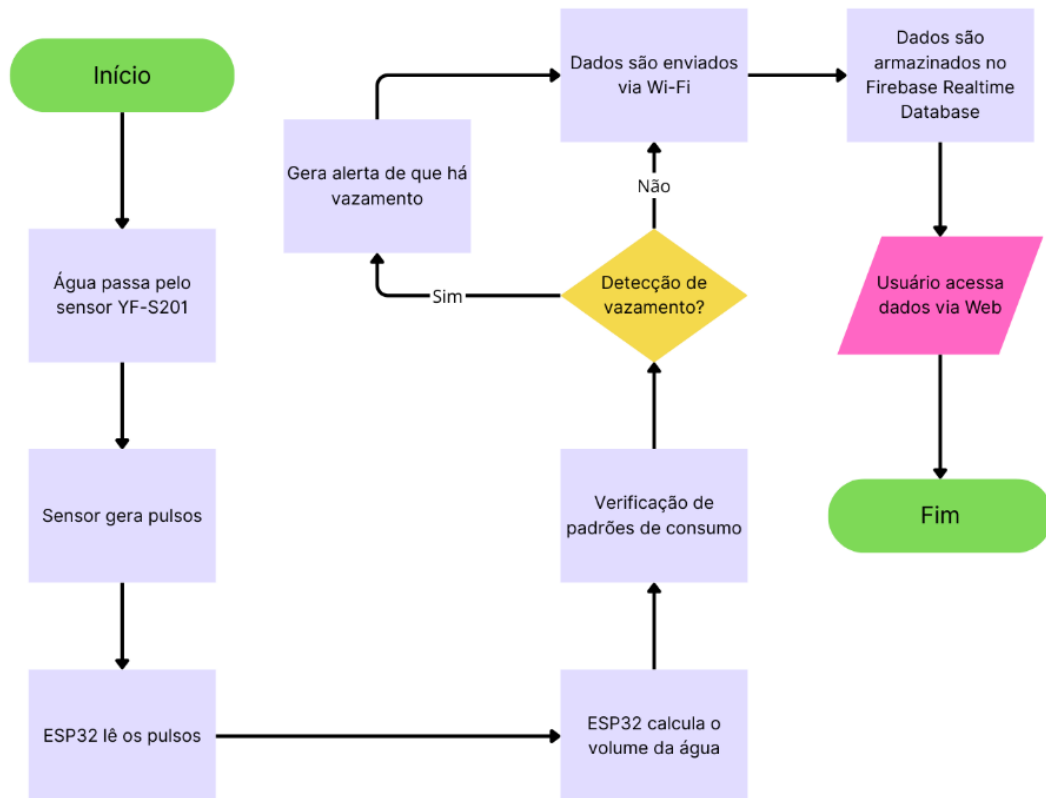


Fonte: Elaborado pela equipe.

Para realizar a leitura da vazão de água no projeto, utilizamos o sensor YF-S201, que possui três fios principais: vermelho, preto e amarelo. Cada um deles tem uma função específica e foi conectado ao ESP32 com base em suas características técnicas. O fio vermelho corresponde à alimentação positiva do sensor e foi conectado ao pino de 5V do ESP32, pois o YF-S201 necessita de uma tensão de operação de 5 volts para funcionar corretamente. O fio preto é o GND (terra) e foi conectado ao pino GND do ESP32, garantindo o referencial elétrico comum entre o sensor e o microcontrolador. Já o fio amarelo é responsável por enviar os pulsos digitais gerados pelo sensor à medida que a água flui por ele. Esses pulsos representam a taxa de vazão e foram conectados ao pino digital GPIO18 do ESP32, que é capaz de interpretar sinais digitais de entrada. Com essa configuração, o microcontrolador consegue contar os pulsos e, a partir disso, calcular a vazão de água em tempo real.

2.1.4.3 Diagrama de Fluxos

Figura 10 - Diagrama de Fluxos.



Fonte: Elaborado pela equipe.

- Início: A água passa pelo sensor de fluxo YF-S201.
- Água passa pelo sensor YF-S201: O sensor detecta o fluxo e gera pulsos digitais com base na rotação de uma hélice interna.
- Sensor gera pulsos: O sensor envia sinais para o ESP32 em forma de pulsos.
- ESP32 lê os pulsos: O ESP32 conta os pulsos emitidos pelo sensor de fluxo.
- ESP32 calcula o volume de água: O ESP32 processa os dados e calcula a quantidade de água consumida com base no número de pulsos recebidos.
- Verificação de padrões de consumo: Após o cálculo, o sistema verifica se os padrões de consumo são normais. Através de uma lógica programada, o ESP32 compara o consumo em tempo real com os padrões definidos.
- Detecção de vazamento: Se o consumo de água continuar constante em horários de inatividade, o sistema identifica como um vazamento.

- Gera alerta de que há vazamento: Quando o sistema interpreta um padrão de consumo anormal, um alerta é gerado e registrado.
- Dados são enviados via Wi-Fi: Os dados de consumo e vazamento são enviados via Wi-Fi para o Firebase Realtime Database.
- Dados são armazenados no Firebase Realtime Database: O Firebase armazena os dados, que podem ser acessados pelo usuário em qualquer momento.
- Usuário acessa dados via Web: O usuário pode acessar os dados de consumo e alertas de vazamento por meio de uma interface web.
- Fim: O processo termina, e o ciclo pode reiniciar à medida que novos dados de consumo de água são coletados e processados.

2.1.4 Cronograma de Execução

Tabela 2 - Cronograma de Execução.

Semana	Atividade	Responsáveis	Status
1	Definição do problema e objetivos do projeto	Toda a equipe	Concluído
2	Levantamento de funcionalidades e requisitos do sistema	Jean e João	Concluído
2	Levantamento dos materiais necessários	Arthur e Stefany	Concluído
3	Simulação do sensor e lógica de leitura no Wokwi	Arthur e Stefany	Concluído
3	Desenvolvimento da lógica de cálculo de volume de água	Jean e Stefany	Concluído
4	Desenvolvimento da lógica de detecção de vazamentos	Arthur e João	Concluído
4	Testes de comunicação Wi-Fi (ESP32)	João e Stefany	Concluído
5	Testes de verificação de padrões de consumo e detecção de vazamentos	Arthur e João	Concluído
5	Testes físicos (sensor de fluxo, ESP32 e atuadores)	Toda a equipe	Concluído

6	Criação dos diagramas e documentação (TDE01)	Toda a equipe	Concluído
6	Integração física dos componentes	Toda a equipe	Concluído
7	Implementação e integração do ESP32 com o Firebase	Jean e Stefany	Concluído
7	Desenvolvimento da interface Web (visualização dos dados)	Arthur e Jean	Concluído
8	Ajustes e otimização do sistema com base nos testes	Arthur e Stefany	Concluído
8	Documentação (TDE02)	Toda a equipe	Concluído
9	Apresentação do projeto	Toda a equipe	Concluído

Fonte: Elaborado pela equipe.

2.1.5 Testes Isolados de Sensores, Atuadores e Módulos

Para garantir a confiabilidade e o bom desempenho do sistema, foram realizados testes isolados dos principais componentes, tanto em ambiente virtual quanto com hardware real. Essa abordagem possibilitou validar o funcionamento individual de sensores, atuadores e módulos de comunicação, assegurando que cada elemento operasse corretamente de forma independente. Com isso, foi possível identificar e corrigir eventuais falhas em fases iniciais do desenvolvimento, o que contribuiu significativamente para uma integração mais eficiente e segura na etapa final do projeto.

1) Testes de Simulação no Ambiente Virtual (Wokwi):

Nesta etapa, foram realizados testes em ambiente virtual utilizando o simulador Wokwi, com o objetivo de validar individualmente os sensores, atuadores e lógicas implementadas no projeto. Os testes buscaram simular o comportamento do sistema, permitindo verificar o correto funcionamento da contagem de pulsos, cálculo de volume de água consumido e detecção de vazamentos antes da integração com os componentes físicos.

A seguir, apresenta-se o código-fonte completo utilizado nas simulações no ambiente virtual Wokwi, que implementa as funcionalidades de contagem de pulsos, cálculo de volume e detecção de vazamentos.

```
#define BUTTON_PIN 4 // Pino do botão
#define PULSES_PER_LITER 5 // Pulsos por litro (ajuste conforme necessário)
#define DEBOUNCE_DELAY 50 // Tempo de debounce em milissegundos

int pulseCount = 0;
float totalLitros = 0.0;
bool lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
int ciclosComConsumo = 0; // Variável para detectar vazamentos

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Botão com pull-up
}

void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);

  // Verifica se o botão foi pressionado e faz debounce simples
  if (buttonState == LOW && lastButtonState == HIGH) {
    if (millis() - lastDebounceTime > DEBOUNCE_DELAY) {
      pulseCount++;
      lastDebounceTime = millis();
    }
  }

  lastButtonState = buttonState;

  // A cada segundo, calcula e mostra os dados
  static unsigned long lastPrint = 0;
  if (millis() - lastPrint >= 1000) {
    float litros = pulseCount / (float)PULSES_PER_LITER;
    totalLitros += litros;

    Serial.print("Pulsos: ");
    Serial.print(pulseCount);
    Serial.print(" | Litros nesta leitura: ");
    Serial.print(litros, 3);
    Serial.print(" | Total acumulado: ");
    Serial.println(totalLitros, 3);

    // Lógica de detecção de vazamento
    if (pulseCount > 0) {
      ciclosComConsumo++;
    } else {
      ciclosComConsumo = 0;
    }

    if (ciclosComConsumo >= 5) { // 5 ciclos seguidos com consumo
      Serial.println("⚠ Vazamento detectado!");
    }

    pulseCount = 0;
    lastPrint = millis();
  }
}
```

1.1) Teste do Sensor de Fluxo (Simulado com Botão no Wokwi):

- **Objetivo:** Simular o funcionamento do sensor de fluxo YF-S201 utilizando um botão no ambiente de simulação Wokwi, com o objetivo de verificar a contagem de pulsos e o cálculo do volume de água em litros.
- **Procedimento/Descrição:** Utilizamos o Wokwi para simular o sensor de fluxo. Em vez do YF-S201, conectamos um botão ao ESP32. Cada clique simulou um pulso de fluxo de água, como seria gerado pelo sensor real.
- **Resultado Esperado:** Ao pressionar o botão no Wokwi, o monitor serial deverá exibir as informações a seguir de forma contínua e atualizada:
 - A quantidade de pulsos detectados no intervalo de 1 segundo
 - A conversão dos pulsos em litros simulados
 - O total acumulado de litros desde o início da execução
 - Alerta de vazamento após 5 segundos de consumo contínuo
- **Resultado Obtido:** O teste foi bem-sucedido. O sistema reconheceu os cliques no botão como pulsos de fluxo, exibindo corretamente os litros e o total acumulado. Também detectou automaticamente o consumo contínuo por 5 ciclos, simulando um vazamento.
- **Trecho do Código-Fonte Utilizado:**

```
#define BUTTON_PIN 4
#define PULSES_PER_LITER 5

int pulseCount = 0;
float totalLitros = 0.0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

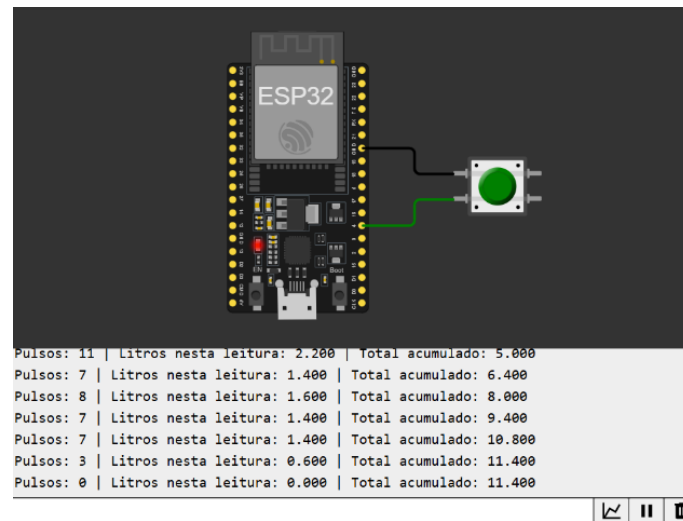
void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);
  if (buttonState == LOW) {
    pulseCount++;
    delay(50);
  }

  static unsigned long lastPrint = 0;
  if (millis() - lastPrint >= 1000) {
    float litros = pulseCount / (float)PULSES_PER_LITER;
    totalLitros += litros;

    Serial.print("Pulsos: ");
    Serial.print(pulseCount);
    Serial.print(" | Litros nesta leitura: ");
    Serial.print(litros, 3);
    Serial.print(" | Total acumulado: ");
    Serial.println(totalLitros, 3);

    pulseCount = 0;
    lastPrint = millis();
  }
}
```

Figura 11 – Simulação do funcionamento do sensor de fluxo utilizando o Wokwi.



Fonte: Elaborado pela equipe com uso do ambiente de simulação Wokwi.

1.2) Teste da Lógica de Cálculo de Volume de Água:

- **Objetivo:** Validar se a fórmula utilizada para o cálculo do volume de água consumido está correta.
- **Procedimento/Descrição:** Foram simulados diferentes números de pulsos utilizando um botão no Wokwi. Cada clique representou um pulso gerado pelo sensor de fluxo YF-S201. Para isso, nós definimos a constante `PULSES_PER_LITER = 5`, ou seja, a cada 5 pulsos, considera-se 1 litro de água consumido.
- **Resultado Esperado:** O monitor serial deve exibir corretamente o volume de água em litros, calculado a partir da fórmula $\text{litros} = \text{pulsos} / \text{PULSES_PER_LITER}$, refletindo a quantidade de pulsos simulados.
- **Resultado Obtido:** O monitor serial exibiu corretamente os valores em litros de acordo com os pulsos simulados. A lógica de conversão está funcionando perfeitamente. A precisão da fórmula foi validada com múltiplos testes em diferentes quantidades de pulsos.
- **Trecho do Código-Fonte Utilizado:**

```
float litros = pulseCount / (float)PULSES_PER_LITER;  
totalLitros += litros;
```

1.3) Teste da Lógica de Detecção de Vazamentos:

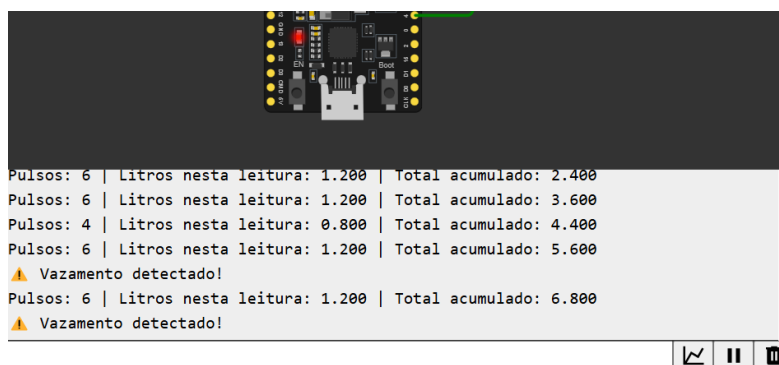
- Objetivo: Verificar se o sistema é capaz de identificar um possível vazamento com base no consumo contínuo de água ao longo do tempo.
- Procedimento/Descrição: Durante a simulação no Wokwi, foi mantido um fluxo contínuo de pulsos (simulados com cliques no botão) por 5 segundos consecutivos. A lógica foi configurada para exibir um alerta de vazamento caso ocorra consumo em 5 ciclos seguidos de 1 segundo.
- Resultado Esperado: Caso ocorra registro contínuo de pulsos durante 5 segundos consecutivos, o sistema deverá exibir no monitor serial um alerta indicando um possível vazamento de água.
- Resultado Obtido: O sistema identificou corretamente o consumo contínuo e exibiu a mensagem “ ⚠ Vazamento detectado!” após 5 ciclos, confirmando que a lógica está funcionando conforme esperado.
- Trecho do Código-Fonte Utilizado:

```
int ciclosComConsumo = 0;

if (pulseCount > 0) {
  ciclosComConsumo++;
} else {
  ciclosComConsumo = 0;
}

if (ciclosComConsumo >= 5) {
  Serial.println("⚠ Vazamento detectado!");
}
```

Figura 9 – Simulação da lógica de detecção de vazamentos no ambiente Wokwi.



Fonte: Elaborada pela equipe com uso do simulador Wokwi.

2) Testes Físicos de Hardware:

Nesta fase, foram realizados testes com os componentes físicos conectados ao ESP32, a fim de validar sensores e comunicação com a nuvem, garantindo que o comportamento da simulação fosse mantido no ambiente real. Foram verificados a leitura dos pulsos do sensor de fluxo com água e o envio dos dados ao Firebase via Wi-Fi.

2.1) Teste do Sensor de Fluxo YF-S201:

- **Objetivo:** Verificar se o sensor de fluxo está funcionando corretamente, contando pulsos com precisão e gerando dados sobre o fluxo de água.
- **Procedimento/Descrição:** O sensor YF-S201 foi conectado ao ESP32 no pino GPIO 14 para a leitura dos pulsos gerados pelo sensor. Uma fonte de água foi utilizada para passar pelo sensor, e a leitura dos pulsos gerados foi monitorada através do monitor serial do Arduino IDE.
- **Resultado Esperado:** O monitor serial deverá mostrar a quantidade de pulsos detectados e calcular corretamente o volume de água, conforme a fórmula usada no sistema.
- **Resultado Obtido:** O teste foi bem-sucedido. O sensor de fluxo YF-S201 foi capaz de contar os pulsos com precisão e calcular o volume de água corretamente.
- **Código-Fonte Utilizado:**

```
#define SENSOR_PIN 14 // Pino conectado ao sensor de fluxo

volatile int pulseCount = 0;

void IRAM_ATTR pulseInterrupt() {
    pulseCount++; // A cada pulso, incrementa o contador
}

void setup() {
    Serial.begin(115200);
    pinMode(SENSOR_PIN, INPUT_PULLUP); // Configura o pino do
    sensor como entrada
    attachInterrupt(digitalPinToInterrupt(SENSOR_PIN),
    pulseInterrupt, FALLING); // Interrupção para contar os pulsos
}

void loop() {
    // Imprime o número de pulsos a cada 1 segundo
    Serial.print("Pulsos: ");
    Serial.println(pulseCount);
    delay(1000); // Aguarda 1 segundo
}
```

2.2) Teste de Comunicação Wi-Fi (ESP32 -> Firebase):

- Objetivo: Validar se o ESP32 consegue se conectar ao Wi-Fi e enviar dados para o Firebase.
- Procedimento/Descrição: Foi feito um teste simples de comunicação, com o ESP32 conectado ao Wi-Fi e enviando dados simulados de consumo de água para o Firebase, visualizados no painel de controle.
- Resultado Esperado: O ESP32 deve se conectar à rede Wi-Fi e enviar os dados de consumo de água para o Firebase corretamente.
- Resultado Obtido: A conexão Wi-Fi foi estabelecida com sucesso e os dados simulados foram enviados para o Firebase sem erros.
- Código-Fonte Utilizado:

```
#include <WiFi.h>
#include <FirebaseESP32.h>

// Credenciais Wi-Fi
const char* ssid = "Visitantes";
const char* password = "";

// Configurações do Firebase
#define FIREBASE_HOST "monitorado-consumo-agua.firebaseio.com"
#define FIREBASE_AUTH ""

// Inicializa o Firebase
FirebaseData firebaseData;

void setup() {
    Serial.begin(115200);

    // Conecta ao Wi-Fi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Conectando ao Wi-Fi...");
    }
    Serial.println("Conectado ao Wi-Fi");

    // Inicializa o Firebase
    Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
}

void loop() {
    // Envia dados simulados para o Firebase
    float consumoAgua = 10.5;

    if (Firebase.setFloat(firebaseData, "/consumoAgua", consumoAgua)) {
        Serial.println("Dados enviados para o Firebase!");
    } else {
        Serial.println("Falha ao enviar dados para o Firebase.");
        Serial.println(firebaseData.errorReason());
    }

    delay(5000);
}
```

2.2 Segunda Etapa (TDE02) – Documentação Técnica

Nesta etapa, foram reunidas e organizadas as informações técnicas do sistema desenvolvido, incluindo a especificação detalhada do funcionamento, o diagrama representativo da arquitetura ciberfísica, o código-fonte devidamente comentado e um relatório de desempenho com base nos testes realizados.

2.2.1 Especificação do Sistema

O sistema tem como finalidade realizar o monitoramento inteligente e em tempo real do consumo de água em ambientes residenciais, utilizando tecnologias embarcadas e conectividade em nuvem. A solução é composta por um microcontrolador ESP32, um sensor de fluxo de água (modelo YF-S201) e integração com a plataforma Firebase Realtime Database para armazenamento e consulta remota dos dados. O projeto foi desenvolvido com foco em acessibilidade, precisão e sustentabilidade, buscando oferecer aos usuários maior controle sobre seu consumo hídrico.

A seguir, estão descritas as funcionalidades principais do sistema:

1) Exibição da Vazão Atual (L/min):

O sistema realiza a leitura contínua do sensor de fluxo, contando os pulsos gerados a cada segundo. A partir dessa contagem, a vazão de água é calculada em litros por minuto (L/min) e exibida em tempo real no monitor serial ou interface remota. Essa funcionalidade permite ao usuário visualizar imediatamente o volume de água sendo utilizado no momento.

2) Histórico de Consumo Total:

Os dados de consumo são acumulados e armazenados no Firebase, permitindo a geração de relatórios com o volume total consumido em diferentes períodos (últimos 60 minutos, últimos 7 dias e últimos 30 dias). Essa funcionalidade auxilia o usuário a acompanhar seu padrão de uso e identificar picos de consumo ao longo do tempo.

3) Cálculo da Vazão Média:

O sistema calcula e disponibiliza a vazão média em diferentes janelas de tempo, com base nos registros armazenados no banco de dados. Essa métrica permite avaliar o comportamento médio de consumo de água da residência e observar variações sazonais ou ocasionais.

4) Identificação do Pico de Vazão:

Durante o monitoramento, o sistema identifica e registra o maior valor de vazão instantânea detectado dentro dos períodos definidos (última hora e dia atual). Esse recurso é útil para identificar eventos de consumo intenso, como o uso simultâneo de chuveiros, torneiras e máquinas de lavar.

5) Indicação do Mês com Maior Consumo:

A aplicação realiza a comparação entre os dados mensais de consumo acumulado, identificando e destacando qual foi o mês com maior volume total de água utilizado. Essa informação ajuda o usuário a entender quais períodos do ano apresentam maior demanda e favorece o planejamento para redução de consumo.

6) Alerta Automático de Vazamento:

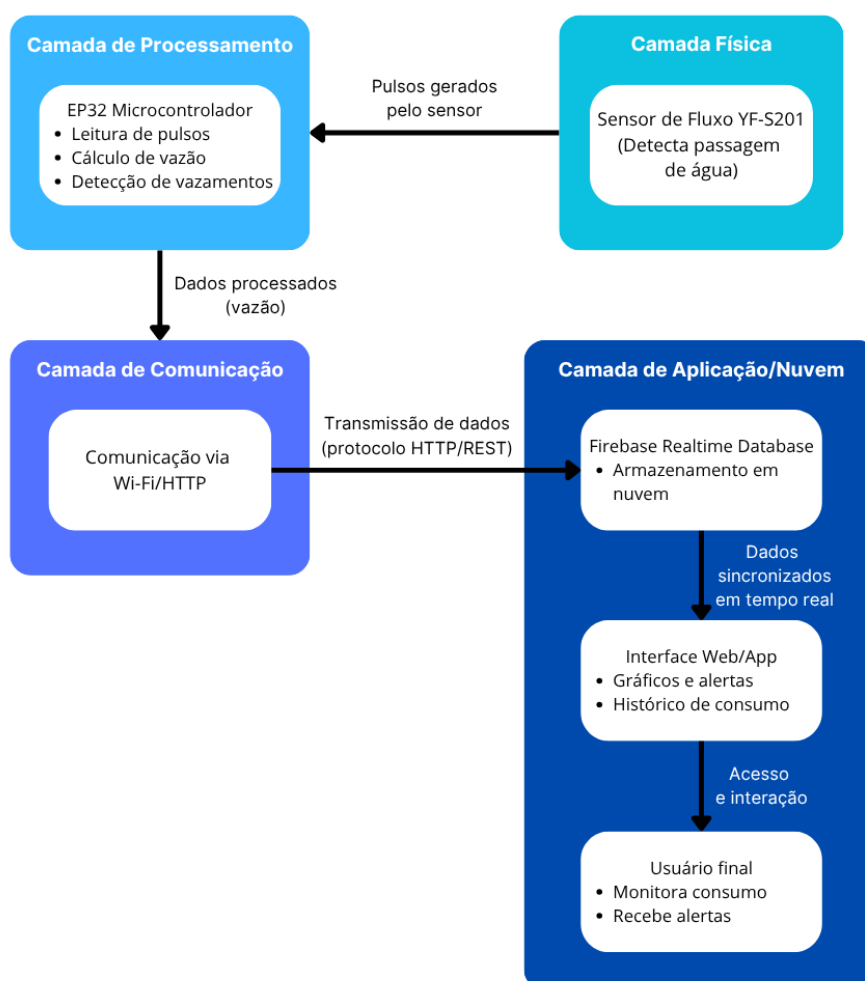
A lógica de detecção de vazamentos no BlueFlow verifica se há um desvio significativo no padrão de consumo recente. A cada nova leitura, o sistema calcula a média de consumo dos últimos 10 minutos e compara com o valor atual. Se a variação entre eles for superior a 50% (ou seja, variação > 0.5), é emitido automaticamente um alerta de possível vazamento.

Essa abordagem baseada em análise temporal permite identificar comportamentos atípicos de consumo em tempo real, mesmo que pequenos, ajudando o usuário a agir rapidamente diante de vazamentos silenciosos — como torneiras pingando, descargas com defeito ou vazamentos em tubulações — prevenindo desperdícios e economizando na conta de água.

2.2.2 Diagrama de Arquitetura Ciberfísica

O sistema desenvolvido foi estruturado seguindo uma arquitetura ciberfísica, organizada em camadas bem definidas, que integram elementos físicos, processamento embarcado, comunicação em rede e aplicação em nuvem. Cada camada possui um papel específico dentro do fluxo de funcionamento do monitoramento de consumo de água, permitindo uma interação contínua entre o ambiente físico e o digital. O diagrama a seguir representa essa estrutura em seus diferentes níveis de atuação.

Figura 10 - Diagrama de Arquitetura Ciberfísica.



Fonte: Elaborado pela equipe.

1. Camada Física: Sensor de Fluxo YF-S201

Esta camada é responsável por captar diretamente os fenômenos do ambiente real. O sensor YF-S201 é instalado na tubulação e detecta o fluxo de água através da rotação de uma hélice interna. Essa rotação gera pulsos elétricos proporcionais ao volume de água que passa pelo sensor, funcionando como o ponto inicial da cadeia de dados do sistema.

2. Camada de Processamento: ESP32 Microcontrolador

Os pulsos emitidos pelo sensor são enviados ao ESP32, que realiza a leitura e o tratamento desses sinais. O microcontrolador calcula a vazão em tempo real, identifica padrões de consumo e verifica possíveis vazamentos com base em variações anormais. Os dados são então organizados e preparados para envio ao servidor remoto.

3. Camada de Comunicação: Wi-Fi com protocolo HTTP/REST

Após o processamento local, o ESP32 realiza a transmissão dos dados por meio de comunicação Wi-Fi. Utilizando o protocolo HTTP e estrutura JSON, os dados são enviados ao Firebase de forma segura e eficiente, garantindo que estejam disponíveis para sincronização e consulta remota.

4. Camada de Aplicação/Nuvem: Firebase Realtime DB + Interface Web/App

Nesta camada, os dados são armazenados em tempo real no Firebase, onde ficam disponíveis para acesso remoto. Por meio de uma interface web responsiva, o usuário final pode visualizar gráficos de consumo, histórico de uso, receber alertas de vazamento e monitorar o sistema em tempo real. Essa camada fecha o ciclo da arquitetura, conectando o ambiente físico ao usuário, com foco em acessibilidade e tomada de decisão.

2.2.3 Código-Fonte Documentado

Esta seção apresenta o código-fonte da solução, com comentários das principais funções e lógicas. O sistema é composto pelo módulo embarcado, que coleta e envia os dados, e pela interface web, que os processa e exibe. A implementação prioriza clareza e integração entre hardware e software.

2.2.3.1 Código para Leitura de Vazão e Envio ao Firebase (monitoramento-consumo-de-agua.ino)

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include "time.h"

#define WIFI_SSID ""
#define WIFI_PASSWORD ""

#define DATABASE_URL "https://blueflow-7e0d7-default-rtdb.firebaseio.com"

#define PINO_SENSOR 18
const float fator_calibracao = 7.5;
volatile int pulsos = 0;
float vazao_Lmin = 0.0;

unsigned long tempoUltimaLeitura = 0;
const unsigned long intervaloLeitura = 1000;

void IRAM_ATTR contaPulso() {
    pulsos++;
}

void conectarWiFi() {
    Serial.print("Conectando ao Wi-Fi");
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
    Serial.println("\n✅ Wi-Fi conectado!");
}

void enviarFirebase(float vazao, String timestamp) {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("❌ Wi-Fi desconectado!");
        return;
    }

    HTTPClient http;
    String url;
    String json = String(vazao, 2);

    url = String(DATABASE_URL) + "/leituras/vazao_Lmin.json";
    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    int httpStatusCode = http.PUT(json);

    if (httpStatusCode > 0) {
        Serial.println("✅ Valor atual enviado!");
    } else {
        Serial.printf("❌ Erro atual: %s\n", http.errorToString(httpStatusCode).c_str());
        http.end();
        return;
    }
    http.end();

    url = String(DATABASE_URL) + "/leituras_com_tempo/" + timestamp + ".json";
    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    httpStatusCode = http.PUT(json);

    if (httpStatusCode > 0) {
        Serial.println("✅ Histórico enviado!");
    } else {
        Serial.printf("❌ Erro histórico: %s\n", http.errorToString(httpStatusCode).c_str());
    }
    http.end();
}
```

```

void sincronizarHorario() {
    configTime(-10800, 0, "pool.ntp.org");
    struct tm timeinfo;
    while (!getLocalTime(&timeinfo)) {
        Serial.print(".");
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
    Serial.println("\n🕒 Horário sincronizado!");
}

String gerarTimestamp() {
    struct tm timeinfo;
    if (!getLocalTime(&timeinfo)) {
        return "erro_timestamp";
    }
    char buffer[20];
    strftime(buffer, sizeof(buffer), "%Y-%m-%d_%H-%M-%S", &timeinfo);
    return String(buffer);
}

void setup() {
    Serial.begin(115200);
    vTaskDelay(300 / portTICK_PERIOD_MS);

    pinMode(PINO_SENSOR, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(PINO_SENSOR), contaPulso, FALLING);

    conectarWiFi();
    sincronizarHorario();
}

void loop() {
    if (millis() - tempoUltimaLeitura >= intervaloLeitura) {
        tempoUltimaLeitura = millis();

        int pulsosPorSegundo = pulsos;
        pulsos = 0;
        vazao_Lmin = (float)pulsosPorSegundo / fator_calibracao;
        Serial.printf("Vazão: %.2f L/min\n", vazao_Lmin);

        String timestamp = gerarTimestamp();
        enviarFirebase(vazao_Lmin, timestamp);
    }
    vTaskDelay(10 / portTICK_PERIOD_MS);
}

```

O código **monitoramento-consumo-de-agua.ino** realiza a leitura do sensor de vazão conectado ao ESP32, calcula a vazão em litros por minuto e envia os dados ao Firebase Realtime Database. As leituras ocorrem a cada segundo, com base na contagem de pulsos gerados pela passagem de água.

Além do valor atual, o sistema também envia os dados com um timestamp gerado via sincronização NTP, garantindo um histórico preciso. O código também gerencia a conexão Wi-Fi e realiza as requisições HTTP para envio ao banco. Esse módulo é essencial para a comunicação entre o hardware e a plataforma web, possibilitando o monitoramento remoto em tempo real.

2.2.3.2 Código JavaScript para Processamento, Atualização e Visualização dos Dados de Vazão (script.js)

```
const firebaseConfig = {
  apiKey: "AIzaSyAlSmPYQo6f2fzR5SVXMNWQd025790WxqY",
  authDomain: "blueflow-7e0d7.firebaseio.com",
  databaseURL: "https://blueflow-7e0d7-default-rtdb.firebaseio.com",
  projectId: "blueflow-7e0d7",
  storageBucket: "blueflow-7e0d7.appspot.com",
  messagingSenderId: "597793620379",
  appId: "1:597793620379:web:b1e3fe56a1cc449119e13e"
};

const app = firebase.initializeApp(firebaseConfig);
const db = firebase.database();

const valorAtualEl = document.getElementById('valorAtual');
const ctx = document.getElementById('graficoVazao').getContext('2d');
let periodoAtual = 'hora';

const grafico = new Chart(ctx, {
  type: 'bar',
  data: {
    labels: [],
    datasets: [{
      label: 'Consumo Total (Litros)',
      data: [],
      backgroundColor: 'rgba(41, 128, 185, 0.6)',
      borderColor: '#2980b9',
      borderWidth: 1,
      borderRadius: 4,
      barThickness: 30
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: { display: true },
      tooltip: { enabled: true },
      title: {
        display: true,
        text: 'Selecione um período',
        font: { size: 16 }
      }
    }
  },
});
```

```

function fecharAlerta() {
  const alertaEl = document.getElementById('alertaVazamento');
  alertaEl.style.display = 'none';
}

function monitorarVazaoAtual() {
  db.ref('leituras/vazao_lmin').on('value', async (snap) => {
    const val = snap.val();
    console.log('Valor atual recebido:', val);

    if (val !== null) {
      const vazaoAtual = parseFloat(val);
      valorAtualEl.textContent = vazaoAtual.toFixed(2) + ' L/min';

      await verificarAnomalia(vazaoAtual);
    } else {
      valorAtualEl.textContent = '--';
    }
  });
}

async function calcularMetricas(dataInicio, calcularPico = false) {
  try {
    const snap = await db.ref('leituras_com_tempo').once('value');
    let soma = 0, count = 0, pico = 0;

    snap.forEach(child => {
      const data = parseTimestampFirebase(child.key);
      const valor = parseFloat(child.val()) || 0;
      if (data >= dataInicio) {
        soma += valor;
        count++;
        if (calcularPico && valor > pico) pico = valor;
      }
    });

    return {
      media: count > 0 ? (soma / count).toFixed(2) : '0.00',
      pico: calcularPico ? pico.toFixed(2) : '--'
    };
  } catch (error) {
    console.error("Erro ao calcular métricas:", error);
    return { media: '0.00', pico: '0.00' };
  }
}

async function atualizarMesMaiorConsumo() {
  const resultado = await calcularMesMaiorConsumo();
  const el = document.getElementById('mesMaiorConsumo');

  if (resultado && resultado.mes && resultado.consumo !== undefined) {
    el.textContent = `${resultado.mes} com ${resultado.consumo} L`;
  } else {
    el.textContent = "dados indisponíveis";
  }
}

async function atualizarTodasMetricas() {
  try {
    const agora = new Date();
    const hoje = new Date(agora);
    hoje.setHours(0, 0, 0, 0);

    const [hora, dia, semana, mes] = await Promise.all([
      calcularMetricas(new Date(agora.getTime() - 3600000), true),
      calcularMetricas(hoje, true),
      calcularMetricas(new Date(agora.getTime() - 604800000)),
      calcularMetricas(new Date(agora.getTime() - 2592000000))
    ]);

    document.getElementById('vazaoMediaHora').textContent = hora.media;
    document.getElementById('vazaoPicoHora').textContent = hora.pico;
    document.getElementById('vazaoMediaDia').textContent = dia.media;
    document.getElementById('vazaoPicoDia').textContent = dia.pico;
    document.getElementById('vazaoMediaSemana').textContent = semana.media;
    document.getElementById('vazaoMediaMes').textContent = mes.media;

    await atualizarMesMaiorConsumo();
  } catch (error) {
    console.error("Erro ao atualizar métricas:", error);
  }
}

```

```

function fecharAlerta() {
  const alertaEl = document.getElementById('alertaVazamento');
  alertaEl.style.display = 'none';
}

function monitorarVazaoAtual() {
  db.ref('leituras/vazao_Lmin').on('value', async (snap) => {
    const val = snap.val();
    console.log('Valor atual recebido:', val);

    if (val !== null) {
      const vazaoAtual = parseFloat(val);
      valorAtualEl.textContent = vazaoAtual.toFixed(2) + ' L/min';

      await verificarAnomalia(vazaoAtual);
    } else {
      valorAtualEl.textContent = '--';
    }
  });
}

async function calcularMetricas(dataInicio, calcularPico = false) {
  try {
    const snap = await db.ref('leituras_com_tempo').once('value');
    let soma = 0, count = 0, pico = 0;

    snap.forEach(child => {
      const data = parseTimestampFirebase(child.key);
      const valor = parseFloat(child.val()) || 0;
      if (data >= dataInicio) {
        soma += valor;
        count++;
        if (calcularPico && valor > pico) pico = valor;
      }
    });

    return {
      media: count > 0 ? (soma / count).toFixed(2) : '0.00',
      pico: calcularPico ? pico.toFixed(2) : '--'
    };
  } catch (error) {
    console.error("Erro ao calcular métricas:", error);
    return { media: '0.00', pico: '0.00' };
  }
}

async function atualizarMesMaiorConsumo() {
  const resultado = await calcularMesMaiorConsumo();
  const el = document.getElementById('mesMaiorConsumo');

  if (resultado && resultado.mes && resultado.consumo !== undefined) {
    el.textContent = `${resultado.mes} com ${resultado.consumo} L`;
  } else {
    el.textContent = "dados indisponíveis";
  }
}

```

```

async function atualizarGrafico(periodo) {
  try {
    const agora = new Date();
    let labels = [], dados = [], cores = [], barThickness = 30, titulo = '';
    const snap = await db.ref('leituras_com_tempo').once('value');
    const leituras = [];

    snap.forEach(child => {
      leituras.push({
        data: parseTimestampFirebase(child.key),
        valor: parseFloat(child.val()) || 0
      });
    });

    if (periodo === 'hora') {
      const horaAtual = new Date(agora);
      horaAtual.setMinutes(0, 0, 0);
      const horaAnterior = new Date(horaAtual.getTime() - 3600000);
      const horaProxima = new Date(horaAtual.getTime() + 3600000);

      labels = ['Hora Anterior', 'Hora Atual', 'Próxima Hora'];
      const valoresPorHora = [0, 0, 0];

      leituras.forEach(({ data, valor }) => {
        const litros = valor / 60;
        if (data >= horaAnterior && data < horaAtual) valoresPorHora[0] += litros;
        else if (data >= horaAtual && data < horaProxima) valoresPorHora[1] += litros;
        else if (data >= horaProxima && data < (horaProxima.getTime() + 3600000))
          valoresPorHora[2] += litros;
      });

      dados = valoresPorHora;
      cores = ['rgba(100, 149, 237, 0.6)', 'rgba(41, 128, 185, 1)', 'rgba(100, 149, 237, 0.6)'];
      barThickness = 50;
      titulo = 'Volume Total por Hora (L)';
    }

    else if (periodo === 'semana') {
      const diasSemana = ['Dom', 'Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb'];
      labels = diasSemana;
      const hoje = new Date(agora);
      const domingo = new Date(hoje.setDate(hoje.getDate() - hoje.getDay()));
      domingo.setHours(0, 0, 0, 0);
      const dadosPorDia = Array(7).fill(0);

      leituras.forEach(({ data, valor }) => {
        const litros = valor / 60;
        if (data >= domingo) {
          const diaIdx = Math.floor((data - domingo) / 86400000);
          if (diaIdx >= 0 && diaIdx < 7) dadosPorDia[diaIdx] += litros;
        }
      });

      dados = dadosPorDia;
      cores = Array(7).fill('rgba(41, 128, 185, 0.8)');
      barThickness = 25;
      titulo = 'Volume Total por Dia da Semana (L)';
    }

    else if (periodo === 'mes') {
      const primeiroDia = new Date(agora.getFullYear(), agora.getMonth(), 1);
      const diasNoMes = new Date(agora.getFullYear(), agora.getMonth() + 1, 0).getDate();
      const semanas = Math.ceil(diasNoMes / 7);

      labels = Array.from({ length: semanas }, (_, i) => `Sem ${i + 1}`);
      const dadosPorSemana = Array(semanas).fill(0);

      leituras.forEach(({ data, valor }) => {
        const litros = valor / 60;
        if (data.getMonth() === agora.getMonth() && data.getFullYear() === agora.getFullYear()) {
          const semanaIdx = Math.floor((data.getDate() - 1) / 7);
          dadosPorSemana[semanaIdx] += litros;
        }
      });

      dados = dadosPorSemana;
      cores = Array(semanas).fill('rgba(52, 152, 219, 0.8)');
      barThickness = 30;
      titulo = 'Volume Total por Semana do Mês (L)';
    }

    else if (periodo === 'anual') {
      labels = ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun', 'Jul', 'Ago', 'Set', 'Out', 'Nov', 'Dez'];
      const dadosPorMes = Array(12).fill(0);

      leituras.forEach(({ data, valor }) => {
        const litros = valor / 60;
        if (data.getFullYear() === agora.getFullYear()) {
          dadosPorMes[data.getMonth()] += litros;
        }
      });

      dados = dadosPorMes;
      cores = Array(12).fill('rgba(20, 90, 120, 0.8)');
      barThickness = 20;
      titulo = 'Volume Total por Mês (L)';
    }

    grafico.data.labels = labels;
    grafico.data.datasets[0].data = dados;
    grafico.data.datasets[0].backgroundColor = cores;
    grafico.data.datasets[0].borderColor = cores.map(c => c.replace('0.8', '1').replace('0.6', '1'));
    grafico.data.datasets[0].barThickness = barThickness;
    grafico.options.plugins.title.text = titulo;
    grafico.update();
  } catch (error) {
    console.error("Erro ao atualizar gráfico:", error);
  }
}

```

```

async function calcularMesMaiorConsumo() {
  try {
    const agora = new Date();
    const snap = await db.ref('leituras_com_tempo').once('value');
    const consumoPorMes = Array(12).fill(0);

    snap.forEach(child => {
      const data = parseTimestampFirebase(child.key);
      const valor = parseFloat(child.val()) || 0;
      if (data.getFullYear() === agora.getFullYear()) {
        consumoPorMes[data.getMonth()] += valor / 60;
      }
    });

    let maiorConsumo = 0;
    let mesMaiorConsumoIdx = 0;
    consumoPorMes.forEach((valor, idx) => {
      if (valor > maiorConsumo) {
        maiorConsumo = valor;
        mesMaiorConsumoIdx = idx;
      }
    });

    const meses = ['Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho',
      'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro'];

    return {
      mes: meses[mesMaiorConsumoIdx],
      consumo: maiorConsumo.toFixed(2)
    };
  } catch (error) {
    console.error("Erro ao calcular mês de maior consumo:", error);
    return { mes: '--', consumo: '0.00' };
  }
}

document.getElementById('btnHora').addEventListener('click', () => {
  periodoAtual = 'hora';
  atualizarTodasMetricas();
  atualizarGrafico(periodoAtual);
  document.querySelectorAll('.btn').forEach(btn => btn.classList.remove('active'));
  document.getElementById('btnHora').classList.add('active');
});

document.getElementById('btnSemana').addEventListener('click', () => {
  periodoAtual = 'semana';
  atualizarTodasMetricas();
  atualizarGrafico(periodoAtual);
  document.querySelectorAll('.btn').forEach(btn => btn.classList.remove('active'));
  document.getElementById('btnSemana').classList.add('active');
});

document.getElementById('btnMes').addEventListener('click', () => {
  periodoAtual = 'mes';
  atualizarTodasMetricas();
  atualizarGrafico(periodoAtual);
  document.querySelectorAll('.btn').forEach(btn => btn.classList.remove('active'));
  document.getElementById('btnMes').classList.add('active');
});

document.getElementById('btnAnual').addEventListener('click', () => {
  periodoAtual = 'anual';
  atualizarTodasMetricas();
  atualizarGrafico(periodoAtual);
  document.querySelectorAll('.btn').forEach(btn => btn.classList.remove('active'));
  document.getElementById('btnAnual').classList.add('active');
});

document.getElementById('btnHora').classList.add('active');
monitorarVazaoAtual();
atualizarTodasMetricas();
atualizarGrafico(periodoAtual);
setInterval(() => {
  atualizarTodasMetricas();
  atualizarGrafico(periodoAtual);
}, 30000);

```

O código **script.js** é responsável pela interface web do BlueFlow, integrando-se ao Firebase Realtime Database para monitorar em tempo real o consumo de água. A partir da configuração inicial com as credenciais do Firebase, a aplicação estabelece a conexão e passa a escutar constantemente a leitura atual da vazão de água em L/min.

Além da exibição do valor instantâneo, o sistema utiliza a biblioteca Chart.js para gerar um gráfico dinâmico, que se adapta de acordo com o período selecionado pelo usuário (hora, semana, mês ou ano). Os dados são organizados e processados conforme o intervalo escolhido, com cores, espessura de barras e títulos personalizados para tornar a visualização mais intuitiva e informativa.

Outra funcionalidade importante é a verificação de anomalias: o sistema compara a vazão atual com a média das leituras dos últimos 10 minutos, e caso haja uma variação significativa, exibe automaticamente um alerta visual para indicar um possível vazamento.

O código também calcula diversas métricas, como média e pico de consumo por hora, dia, semana e mês, além de identificar qual foi o mês com maior volume registrado ao longo do período monitorado. Essas informações são atualizadas automaticamente a cada 30 segundos, garantindo que o usuário tenha sempre acesso a dados recentes e confiáveis.

Toda a lógica é escrita de forma modular, com funções bem definidas para facilitar a manutenção, reutilização e compreensão do código. Essa abordagem torna o sistema mais robusto e preparado para evoluções futuras.

2.2.3.3 Código da Interface Web para Visualização de Consumo (index.html)

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1"/>
    <title>BlueFlow</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter&display=swap" rel="stylesheet"/>
    <link href="style.css" rel="stylesheet"/>
  </head>
  <body>
    <div class="dashboard">
      <div class="title">
        <h1>💧 BlueFlow - Monitoramento de Vazão 💧</h1>
      </div>

      <div class="title-mobile">
        <h1></h1>BlueFlow<br>Monitoramento de Vazão</h1>
      </div>

      <div class="metrics-top" id="alertaVazamento">

        ⚠️ Atenção: possível vazamento detectado! ⚠️<br><br>
        Vazão atual: <span id="valorAtualAlerta"></span> L/min<br>
        Média (últimos 10 min): <span id="mediaAlerta"></span> L/min<br><br>

        <button id="fecharAlertaVazamento" onclick="fecharAlerta()">Fechar</button>
      </div>

      <div class="metrics-top">
        <div class="metric-top">
          <h2 id="valorAtual">--</h2>
          <p>Vazão Atual (L/min)</p>
        </div>
        <div class="icon">
          
        </div>
      </div>

      <h2 class="canvas-title">Histórico de Vazão de Água</h2>

      <div class="canvas-container">
        <canvas id="graficoVazao"></canvas>
      </div>

      <div class="filtro-tempo">
        <button id="btnHora" onclick="trocarPeriodo('hora')" class="active">Última Hora</button>
        <button id="btnSemana" onclick="trocarPeriodo('semana')">Última Semana</button>
        <button id="btnMes" onclick="trocarPeriodo('mes')">Último Mês</button>
        <button id="btnAnual" onclick="trocarPeriodo('anual')">Último Ano</button>
      </div>

      <h2 class="subtitle">Mais informações: </h2>

      <h2 class="subtitle-infos">Última hora: </h2>

      <div class="metrics-bottom">
        <div class="metric-bottom">
          <h2 id="vazaoMediaHora">--</h2>
          <p>Vazão Média Última Hora (L/min)</p>
        </div>
        <div class="metric-bottom">
          <h2 id="vazaoPicoHora">--</h2>
          <p>Pico de Vazão Última Hora (L/min)</p>
        </div>
      </div>
    </div>
```

```

<h2 class="subtitle-Infos"> Hoje: </h2>

    <div class="metrics-bottom">
        <div class="metric-bottom">
            <h2 id="vazaoMediaDia">--</h2>
            <p>Vazão Média Hoje (L/min)</p>
        </div>
        <div class="metric-bottom">
            <h2 id="vazaoPicoDia">--</h2>
            <p>Pico de Vazão Hoje (L/min)</p>
        </div>
    </div>

<h2 class="subtitle-Infos"> Média Semanal e Mensal:</h2>

    <div class="metrics-bottom">
        <div class="metric-bottom">
            <h2 id="vazaoMediaSemana">--</h2>
            <p>Vazão Média Última Semana (L/min)</p>
        </div>
        <div class="metric-bottom">
            <h2 id="vazaoMediaMes">--</h2>
            <p>Vazão Média Último Mês (L/min)</p>
        </div>
    </div>

<h2 class="subtitle-Infos"> Mês com maior consumo: </h2>

    <div class="metric-bottom mesMaiorConsumo">
        <div class="mesMaiorConsumo-text">
            <h2 id="mesMaiorConsumo">--</h2>
            <p>Mês com Maior Consumo total (Litros)</p>
        </div>
        <div class="icon-grafico">
            
        </div>
    </div>

</div>

<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-database-compat.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chartjs-adapter-date-fns"></script>
<script src="script.js"></script>
</body>
</html>

```

O código **index.html** é responsável pela criação da interface do BlueFlow, a página exibe um título responsivo, um alerta visual que aparece ao detectar possível vazamento com dados da vazão atual e média, além da vazão atual em tempo real. Também inclui um gráfico interativo que mostra o histórico de consumo em diferentes períodos, como última hora, semana, mês e ano. O usuário pode acompanhar estatísticas detalhadas, como médias e picos de vazão por hora, dia, semana e mês, além de visualizar o mês com maior consumo. O layout foi pensado para facilitar o acompanhamento rápido e intuitivo dos dados de consumo de água.

2.2.3.4 Código para Estilização da Interface Web (style.css)

```
body {
  font-family: 'Inter', sans-serif;
  background-color: rgba(235, 235, 235, 0.288);
  padding: 20px;
  color: #2c3e50;
  display: flex;
  flex-direction: column;
  align-items: center;
  min-height: 100vh;
}
h1 {
  margin-bottom: 10px;
}
.dashboard {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  width: 100%;
  max-width: 900px;
  background: #e7e9ff;
  border-radius: 10px;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.253);
  padding: 20px;
}
.title{
  font-size: 1.1rem;
}
.title-mobile{
  display: none;
}
#alertaVazamento {
  display: none;
  width: 80%;
  padding: 20px 30px;
  margin-bottom: 15px;
  border-radius: 12px;
  background-color: #f8d7da;
  color: #842029;
  font-weight: bold;
  font-size: 22px;
  text-align: center;
  transition: all 0.5s ease;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.253);
}
#fecharAlertaVazamento {
  margin-top: 10px;
  padding: 8px 15px;
  border: none;
  background-color: #97262f;
  color: #f8d7da;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}
#fecharAlertaVazamento:hover {
  background-color: #6a1b1e;
}
```

```

.metrics-top {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin: 50px;
  margin-bottom: 30px;
  width: 90%;
}
.metric-top {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: flex-start;
  width: 50%;
  height: 70%;
  background: #ffffff;
  border-radius: 12px;
  padding: 20px 30px;
  text-align: center;
  font-size: 23px;
  flex: 1;
  margin: 0 10px;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.253);
}
.icon{
  display: flex;
  justify-content: center;
  width: 50%;
}
.icon img {
  width: 70%;
}
.subtitle{
  font-size: 30px;
  margin-top: 60px;
  margin-bottom: 40px;
}
.subtitle-infos{
  margin-top: 0;
  margin-right: auto;
  padding-left: 33px;
}
.metrics-bottom{
  display: flex;
  justify-content: center;
  align-items: center;
  margin-bottom: 30px;
  width: 95%;
}
.metric-bottom {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: flex-start;
  width: 50%;
  height: 100%;
  background: #ffffff;
  border-radius: 12px;
  padding: 20px 30px;
  text-align: center;
  font-size: 20px;
  flex: 1;
  margin: 0 10px;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.253);
}

```

```

.mesMaiorConsumo {
  display: flex;
  flex-direction: row;
  align-self: flex-start;
  margin-left: 30px;
  margin-bottom: 50px;
  width: 86%;
}
.mesMaiorConsumo-text{
  width: 60%;
  display: flex;
  flex-direction: column;
  align-self: center;
  text-align: start;
  margin-left: 2%;
}
.icon-grafico {
  display: flex;
  justify-content: center;
  align-items: center;
  width: 40%;
}
.icon-grafico img {
  width: 90%;
  height: auto;
}
.metric h2 {
  margin: 0;
  font-size: 2rem;
  color: #1a32a0;
}
.metric p {
  margin: 5px 0 0;
  font-weight: 600;
}
.filtro-tempo {
  margin: 20px 0;
  text-align: center;
}
.filtro-tempo button {
  background-color: #3352dd;
  color: white;
  border: none;
  padding: 10px 16px;
  margin: 0 5px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
}
.filtro-tempo button:hover {
  background-color: #142881;
}
.canvas-title{
  font-size: 30px;
}
.canvas-container{
  display: flex; height: 380px !important;
}

```

```

@media (max-width: 600px) {
  .dashboard {
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
    flex-direction: column;
    width: 100%;
  }
  .title{
    display: none;
  }
  .title-mobile{
    display: flex;
    text-align: center;
    font-size: 28px;
    font-weight: bold;
  }
  .icon{
    display: none;
  }
  .canvas-title {
    font-size: 22px;
    text-align: center;
    margin-bottom: 10px;
  }
  .canvas-container {
    flex-direction: column;
    margin: 10px;
    padding: 15px;
  }
  canvas {
    width: 100% !important;
    height: 300px !important;
  }
  .filtro-tempo {
    display: grid;
    grid-template-columns: 1fr 1fr;
    grid-gap: 8px;
    width: 90%;
    padding: 0 10px;
  }
  .filtro-tempo button {
    min-width: auto;
    padding: 12px 8px;
    font-size: 14px;
  }
  .subtitle{
    font-size: 25px;
  }
  .subtitle-infos{
    margin-top: 0;
    margin-right: 0;
    padding-left: 0;
    font-size: 20px;
  }
  .metric-top{
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
  }
  .metrics-bottom {
    flex-direction: column;
  }
  .metric-bottom {
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
    width: 80%;
    margin-bottom: 30px;
  }
  .mesMaiorConsumo {
    font-size: 16px;
    width: 80%;
    margin-left: 10px;
    margin-bottom: 50px;
  }
  .mesMaiorConsumo-text{
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    text-align: center;
    margin-left: 0;
  }
  .icon-grafico{
    display: none;
  }
}

```

O código **style.css** é responsável por definir toda a aparência visual da aplicação. Ele organiza o layout, os espaçamentos, cores, fontes e tamanhos dos elementos para tornar a interface clara, moderna e fácil de usar. Além disso, o CSS garante que o site seja responsivo, adaptando-se bem a diferentes tamanhos de tela, como computadores, tablets e celulares. Com o CSS, conseguimos destacar informações importantes, criar uma hierarquia visual agradável e melhorar a usabilidade do monitoramento de vazão, deixando a experiência do usuário mais intuitiva e agradável.

2.2.4 Relatório de Desempenho e Testes

Com o objetivo de verificar o desempenho e a confiabilidade do sistema, foram realizados testes em diversos cenários, contemplando condições normais, exceções e situações críticas. Avaliou-se a estabilidade das leituras, a detecção de anomalias, a comunicação com a nuvem e a resposta a reinicializações e variações de fluxo. A Tabela 3 apresenta um resumo dos principais testes realizados, com seus respectivos resultados esperados e obtidos.

Tabela 3 – Relatório de Testes realizados

Cenário	Descrição	Resultado esperado	Resultado Obtido
Inicialização do sistema	Reinicializar o ESP32 com conexão Wi-Fi e sensor.	Sistema reinicia com envio normal de dados após boot.	Reinicialização sem falhas, leituras retomadas normalmente.
Leitura estável	Fluxo contínuo dentro do padrão normal.	Leituras estáveis, sem alertas.	Leituras consistentes, sem alertas falsos.
Pico de vazão	Simular um uso simultâneo de múltiplos pontos de água.	Identificação do maior valor de vazão no período.	Pico registrado corretamente e exibido na interface.
Variação simulada	Aumento abrupto do fluxo de água.	Alerta de possível vazamento ativo.	Alerta exibido corretamente no dashboard.

Sem fluxo	Interrupção repentina total do fluxo de água.	Leituras próximas a zero e Alerta de possível vazamento ativo.	Leituras zeradas exibidas com precisão.
Desconexão de rede	Desligar Wi-Fi do ESP32.	Pausa na atualização dos dados.	Dados pausados no Firebase conforme esperado.
Reconexão da rede	Reestabelecer conexão Wi-Fi.	Retomada da captura e exibição de dados.	Retorno das leituras e gráficos atualizados normalmente.
Troca de períodos	Alterar filtros do gráfico (hora, semana, mês, ano).	Gráficos atualizados com dados corretos.	Filtragem e atualização do gráfico funcionando sem erros.

Fonte: Elaborado pela equipe.

3 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema de monitoramento de consumo de água utilizando o microcontrolador ESP32 mostrou-se eficaz e viável para promover o uso consciente da água em ambientes residenciais. A integração entre o sensor de fluxo e a plataforma Firebase permitiu a coleta, o processamento e o armazenamento de dados em tempo real, facilitando o acesso remoto e o acompanhamento do consumo.

Os testes realizados, tanto na simulação quanto no hardware real, confirmaram a precisão na medição do volume de água e a capacidade do sistema de detectar padrões anormais de consumo, possibilitando a identificação de vazamentos. Isso representa uma contribuição importante para a sustentabilidade, pois permite que os usuários tenham maior controle sobre seu uso hídrico e possam agir rapidamente diante de desperdícios.

Além disso, a arquitetura ciberfísica desenvolvida demonstra o potencial das tecnologias embarcadas associadas à computação em nuvem para a criação de soluções inteligentes e acessíveis para problemas cotidianos. O projeto abre espaço para futuras melhorias, como a inclusão de notificações automáticas, otimização da interface de visualização e a expansão para monitoramento de múltiplos pontos de consumo.

Por fim, destaca-se a importância de iniciativas tecnológicas voltadas à preservação dos recursos naturais, especialmente em um cenário de crescente demanda e escassez hídrica. O sistema apresentado contribui para a conscientização dos usuários e para o incentivo a práticas mais responsáveis, alinhando tecnologia e sustentabilidade de forma prática e eficiente.

4 LINKS (REPOSITÓRIO GITHUB E SIMULAÇÃO WOKWI)

Nesta seção, encontram-se os links para o repositório do projeto no GitHub, que contém todo o código-fonte e a documentação completa, além da simulação realizada na plataforma Wokwi. A simulação permite a visualização e o teste virtual do funcionamento do sistema, facilitando a compreensão da lógica implementada antes da montagem física.

- Link do repositório no GitHub:
<https://github.com/arthurpansera/Projeto-ESP32.git>
- Link da simulação no Wokwi:
<https://wokwi.com/projects/428941867633477633>

5 REFERÊNCIAS BIBLIOGRÁFICAS

ADAFRUIT. **RTCLib.h**. GitHub, 2025. Disponível em:

<<https://github.com/adafruit/RTCLib>>. Acesso em: 22 abr. 2025.

ARDUINO COMMUNITY. **Ticker.h**. Arduino, 2025. Disponível em:

<<https://www.arduino.cc/en/Reference/Ticker>>. Acesso em: 22 abr. 2025.

ARDUINO COMMUNITY. **WiFi.h**. Arduino, 2025. Disponível em:

<<https://www.arduino.cc/en/Reference/WiFi>>. Acesso em: 22 abr. 2025.

ARDUINO COMMUNITY. **Wire.h**. Arduino, 2025. Disponível em:

<<https://www.arduino.cc/en/Reference/Wire>>. Acesso em: 22 abr. 2025.

BIT A BIT. **Arduino - Usando o módulo relógio de tempo real DS1307**.

Disponível em: <<https://youtu.be/yktnhH69-K8?si=AQbZ21Y9afA9ial0>>.

Acesso em: 12 abr. 2025.

CASA DAS CORREIAS E BORRACHAS. **Mangueira jardim siliconada amarela 1/2 elite 35 metros**. Disponível em:

<<https://www.casadascorreiasborrachas.com.br/produto/mangueira-jardim-siliconada-amarela-12-elite-35-metros.html>>. Acesso em: 18 jun. 2025.

CONFEDERAÇÃO NACIONAL DE MUNICÍPIOS. **Brasileiro consome em média 154 litros de água por dia, aponta ONU**. Disponível em:

<<https://cnm.org.br/comunicacao/noticias/brasileiro-consome-em-media-154-litros-de-agua-por-dia-aponta-onu>>. Acesso em: 14 jun. 2025.

ELETRUS COMP. **Sensor fluxo de água 1/2" YF-S201**. Disponível em:

<<https://www.eletruscomp.com.br/produto/sensor-fluxo-de-agua-1-2-yf-s201/>>.

Acesso em: 18 jun. 2025.

MAKER HERO. **Jumpers macho-fêmea x40 unidades**. Disponível em:
<<https://www.makerhero.com/produto/jumpers-macho-femea-x40-unidades/>>.
Acesso em: 18 jun. 2025.

MAKER HERO. **Protoboard 400 pontos**. Disponível em:
<<https://www.makerhero.com/produto/protoboard-400-pontos/>>.
Acesso em: 18 jun. 2025.

MOBIZT. **FirestoreESP32.h**. GitHub, 2025. Disponível em:
<<https://github.com/mobizt/Firebase-ESP32>>. Acesso em: 22 abr. 2025.

OPENAI. **ChatGPT**. San Francisco: OpenAI, 2025. Disponível em:
<<https://chat.openai.com>>. Acesso em: 8 abr. 2025.

TCJ ELETRÔNICA. **Medidor de Litros com Arduino e Sensor de Fluxo YF-S201**. Disponível em: <<https://www.youtube.com/watch?v=gDUVdljgVlk>>.
Acesso em: 12 abr. 2025.

TRATA Brasil. **Perdas de água no Brasil aumentam em 2024**. Disponível em:
<<https://tratabrasil.org.br/perdas-de-agua-2024/>>. Acesso em: 18 jun. 2025.

TRATA Brasil. **Redução de perdas de água poderia beneficiar mais de 21 milhões de brasileiros, mais que a população total do Chile**. 20 set. 2024.
Disponível em: <<https://tratabrasil.org.br/perdas-reducao-mais-de-21-milhoes-agua/>>. Acesso em: 17 jun. 2025.

VICTOR VISION. **Placa ESP32: o que é, como funciona e para que serve**.
Disponível em: <<https://victorvision.com.br/blog/placa-esp32/>>.
Acesso em: 18 jun. 2025.

WOKWI. **Simulação de circuitos online**. Disponível em: <<https://wokwi.com/>>.
Acesso em: 20 abr. 2025.